Informe de pruebas y cobertura

Master Martini | DSY1104 - EP2 | 22 de octubre de 2026

1. Objetivo, herramientas y alcance

Se verificaron comportamientos observables del frontend, persistencia local y servicios de demostración. Se utiliza Vitest 5.0.5, React Testing Library, jsdom y cobertura V8; Node para lógica y API; Playwright con Chrome para recorridos reales. Vite 7.3.7 compila la aplicación. Las pruebas de frontend se ejecutaron con Vitest.

2. Resultados comprobados

Comando	Resultado	Evidencia en testing/evidence/
npm ci	Correcto	Instalación desde package-lock.json
npm run check:syntax	Correcto	check-syntax.txt
npm run check:logic	16/16	check-logic.txt
npm run test:node	39/39	test-node.txt
npm run test:assets	9/9	test-assets.txt
npm run test:api	15/15	test-api.txt
npm test	62/62	test-vitest.txt
npm run test:coverage	62/62	test-coverage.txt
npm run build	Correcto	build.txt
npm run test:e2e	16/16 escenarios	browser/results.json y browser-run.txt
npm run test:e2e:api	5/5 con API real	backend-browser/results.json

La suite Vitest contiene 62 casos, incluidos 13 controles básicos. Son 44 pruebas de componentes o vistas, nueve del hook de contexto y nueve de funciones, datos o almacenamiento. Node, recursos, API y E2E son suites complementarias: sus recuentos no se presentan como pruebas unitarias de componentes.

3. Cobertura real y comparación

Métrica	Base inicial	Base con alcance completo	Resultado final
Sentencias	38,40 %	22,13 %	95,28 %
Ramas	34,83 %	23,14 %	90,55 %
Funciones	37,91 %	19,54 %	96,60 %
Líneas	45,54 %	25,00 %	97,56 %

La base inicial instrumentaba únicamente archivos importados y reportaba 184/404 líneas. El alcance completo considera `src/**/*.{js,jsx}`, incluso archivos no ejecutados, con excepción de `src/tests/**` y `main.jsx`, que inicializa el DOM. Con ese alcance, las pruebas de base ejecutaban 184/736 líneas (25,00 %) y la suite completa ejecuta 720/738 (97,56 %). La diferencia de dos líneas corresponde a la sincronización del historial tras restaurar los datos de la API. CSS no se incluye en el cómputo de cobertura JavaScript.

4. Módulos representativos

Módulo	Líneas	Interpretación
ProductMedia.jsx	100 %	Medición V8
SiteLayout.jsx	92.3 %	Medición V8
ShopContext.jsx	100 %	Medición V8
AccountAdmin.jsx	94.48 %	Medición V8
CartCheckout.jsx	100 %	Medición V8
Clases.jsx	100 %	Medición V8
ShopPages.jsx	97.67 %	Medición V8
commerce.js	96.55 %	Medición V8
storage.js	100 %	Medición V8

Home.jsx, App.jsx, ErrorBoundary.jsx y ShopContext.jsx alcanzan 100 % de líneas. Se prueban temporizadores, pausa por foco, pestaña oculta, movimiento reducido, rutas, errores y contrato API. La cobertura E2E no se suma a V8. Las ramas restantes incluyen variantes de datos y recursos opcionales; un porcentaje alto no demuestra ausencia absoluta de errores.

5. Casos nuevos de componentes: C01-C13

ID	Comportamiento verificado	Resultado
C01	actualiza precio, descuento, enlace y callback al cambiar props	Aprobado
C02	utiliza respaldos tras errores de imagen y se recupera con nuevas props	Aprobado
C03	busca, combina categoría y recupera resultados vacíos	Aprobado
C04	ordena precios con descuento y nombres sin alterar el catálogo	Aprobado
C05	las ofertas excluyen productos sin descuento	Aprobado
C06	detalle limita cantidad al stock y actualiza el contador compartido	Aprobado
C07	menú móvil y búsqueda navegan y cierran sus estados	Aprobado
C08	cambia cantidades, totales, stock y elimina el último artículo	Aprobado
C09	vacía el carrito completo	Aprobado
C10	bloquea campos inválidos y limpia comuna al cambiar región	Aprobado
C11	precarga cliente, aplica retiro y aprueba guardando pedido y stock	Aprobado
C12	rechazo conserva carrito y stock sin crear un pedido	Aprobado
C13	impide confirmar stock insuficiente y carrito vacío	Aprobado

6. Ejemplo de prueba con estado compartido

C11 monta Checkout dentro de MemoryRouter y ShopProvider, con productos y perfil conocidos en localStorage. Selecciona retiro y confirma el pedido. Luego espera la vista de éxito y verifica que el pedido cuesta $9.000, pertenece al cliente, el stock baja de 2 a 1 y el carrito queda vacío. Se usa checkout real: no se reemplaza ShopContext por un mock.

C12 selecciona pago rechazado y verifica el caso alternativo: ningún pedido, stock igual a 2 y carrito conservado. C10 valida los errores asociados a los campos y que el formulario no navegue con datos incompletos.

7. Casos nuevos de componentes: C14-C26

ID	Comportamiento verificado	Resultado
C14	login rechaza credenciales incorrectas y acepta perfil cliente	Aprobado
C15	login administrador navega al panel y no persiste contraseña	Aprobado
C16	registro valida, confirma contraseña y guarda perfil sin contraseña	Aprobado
C17	protege administración contra visitantes y clientes	Aprobado
C18	CRUD valida, crea, edita y elimina con persistencia	Aprobado
C19	vendedor puede editar pero no ve eliminar; consulta pestañas	Aprobado
C20	cuenta filtra pedidos por cliente y permite cerrar sesión	Aprobado
C21	contacto no confirma campos inválidos y confirma datos válidos	Aprobado
C22	suscripción cambia a confirmación local	Aprobado
C23	inscribe una sola vez, persiste cupos y muestra clase en Mi cuenta	Aprobado
C24	agrega todos los insumos y respeta stock al repetir	Aprobado
C25	recupera datos inválidos de almacenamiento sin pantalla vacía	Aprobado
C26	fallo de API bloquea checkout sin mutar stock ni pedidos	Aprobado

8. Mocks y aislamiento

C01 usa vi.fn para observar el callback onAdd y sus argumentos. C18 usa vi.spyOn(window, confirm) para probar cancelación y aceptación de eliminación. C26 usa vi.stubGlobal(fetch) para provocar un error de red y verificar que el checkout no modifique pedidos ni stock. El caso 12 simula una respuesta API válida. No se simulan los componentes ni las funciones comerciales.

setup.js limpia localStorage antes de cada prueba y desmonta componentes, restaura espías, retira globals simulados y elimina la bandera de API después. Las consultas usan roles y etiquetas; fireEvent representa cambios y clics. findBy y waitFor sincronizan resultados asíncronos.

9. Verificación funcional y responsive

Los 16 escenarios E2E cubren inicio y rutas en 390, 768, 1366, 1440 y 1920 px; búsqueda, filtro, detalle, carrito, cantidades y eliminación; checkout inválido, rechazo y aprobación; login y CRUD con recarga; clases, ingredientes, cuenta e historial; sesión aislada y carga de imágenes. Clases, carrito con productos y checkout también se revisan en 390, 768, 1366 y 1920 px. Se comprueba ausencia de desbordamiento horizontal. Las capturas actuales se guardan en testing/evidence/browser/.

A09 exige `/assets/Portada.jpg` y verifica los tres recursos del carrusel. El archivo está presente y el navegador lo carga. Las esperas E2E verifican la actualización del filtro antes de medir tarjetas. La función `refreshCatalogProductMedia` mantiene compatibilidad con referencias fotográficas almacenadas localmente.

10. Reproducibilidad y evidencias

Desde la raíz: npm ci; npm run check:syntax; npm run check:logic; npm run test:node; npm run test:assets; npm run test:api; npm test; npm run test:coverage; npm run build. Para E2E: iniciar npm run dev en una terminal y npm run test:e2e en otra; requiere Chrome instalado. coverage/index.html muestra el detalle V8.

`baseline-coverage.txt/json` y `baseline-full-scope/coverage-summary.json` documentan la base de comparación. `validation.json` reúne los códigos de salida; `browser/` contiene las capturas de los escenarios funcionales.

11. Conclusiones y límites

Las pruebas verificadas pasan y el build genera dist/. La suite aporta ejemplos de renderizado, props, estado, eventos, DOM, persistencia y errores. La cobertura de ramas es 90,55 % y la de líneas 97,56 %. El modo API se validó con cinco escenarios reales de navegador: carga, compra e historial, CRUD, reset y caída del servidor. El error ECONNREFUSED en el último escenario es intencional y verifica la comunicación del fallo. La revisión responsive automatizada y visual no es una auditoría integral de accesibilidad.

La evaluación del trabajo y su defensa depende del docente. La presentación individual representa el 60 % y requiere que cada integrante explique las decisiones, pruebas y límites del prototipo. El porcentaje de cobertura no determina por sí solo una nota.

12. Ampliación de pruebas y regresión corregida

Casos	Objetivo	Resultado
H01-H06	Carrusel automático y manual; foco, visibilidad, movimiento reducido, filtros, promoción, cuenta regresiva y diálogo.	6/6
H07-H10	Rutas de App, artículos, página inexistente, IntersectionObserver, volver arriba y ErrorBoundary con recarga.	4/4
P01-P07	Contrato API del ShopContext: CRUD, checkout, error HTTP, JSON inválido, validaciones, reset y cuota de almacenamiento. P06 usa tres entradas inválidas.	9/9
F01-F04	Todos los campos de producto, cancelación/reset, corrección del checkout, error remoto de stock y pedido inexistente.	4/4

P05 verifica que `/api/reset` sincronice el estado de pedidos de `ShopContext` después de restaurar los datos remotos. La confirmación administrativa informa el restablecimiento y B04 comprueba la actualización sin recargar la página. El modo local mantiene su comportamiento propio de historial.

13. Umbrales y automatización de calidad

vite.config.js exige al menos 95 % de líneas, sentencias y funciones, y 90 % de ramas. npm run validate:all reúne sintaxis, lógica, recursos, API, cobertura y compilación. Se comprobó el rechazo del control elevando temporalmente por línea de comandos la exigencia a 100 %: las 62 pruebas pasaron, pero el comando terminó con código 1 por cobertura insuficiente. El registro coverage-gate-check.txt documenta este resultado esperado; la configuración entregada conserva 95/90.

El workflow .github/workflows/quality.yml configura npm ci, validate:all y E2E con API real en GitHub Actions, además de guardar evidencias. La configuración de integración continua está incluida. Los resultados de este informe corresponden a ejecuciones locales, no a una ejecución remota de GitHub Actions.

14. Reproducción del backend aislado

npm run test:e2e:api inicia Vite y la API en puertos temporales, con un directorio de datos exclusivo. Utiliza la aplicación real con VITE_USE_BACKEND=true y verifica resultados también mediante

HTTP. Cierra sus procesos y elimina únicamente sus datos temporales. No modifica server/.data. Las capturas y solicitudes quedan en testing/evidence/backend-browser/.